Algorithms

*a step by step procedure for solving a computation problem

Algorithm

- written at design time
- domain knowledge
- any language since its concept
- hardware & software
- after creating it is analyzed

Program

- for implementation using syntax
- programmer
- need to use program language
- depended on hardware & operating system
- run the program / testing

Priori Analysis)

* the study of an algorithm in greater detail, knowing how it is working, and reviewing results.

1) Algorithm
2) Independent of Language
3) Hardware Independent
4) Time & Space Function

Posteriori Testing)

* testing / running of a program to analyze watchtime and memory / bytes

1) Program
2) Language Dependent
3) Hardware Dependent
4) Watch-time & Bytes

Characteristics of an Algorithm:

1) Input * can take zero or more inputs

2) Output * must generate some type of result

3) Definiteness * every statement must be clear, and use real values

4) Finiteness * must stop; whether that be after completing or manually like a server.

5) Effectiveness * don't write unnecessary statements, keep it on topic

---

How to analyze an Algorithm:

1) Time * want to be time efficient.

2) Space * how much memory space it will consume.

3) Network Consumption * how much data transfer is done in algorithm

4) Power

5) CPU Registers * how much CPU memory (registers) is it consuming

```
def algoSwap(a,b):

    temp = a
    a = b
    b = temp
```

example algorithm as Python function

space
a → 1
b → 1
temp → 1
s(n) = 3

```
def algoSwap(a,b):

    temp = a   → 1 unit of time
    a = b      → 1
    b = temp   → 1
```

3

f(n) = 3 or constant

Frequency Count Method:

```
def algorithmSum(a, n):
```

$a =$

8	3	7	5	9
0	1	2	3	4

$n = 5$

```
    s = 0 → 1
    for i in range(n): → n+1
        s = s + a[i] → n
```

$i = 0 \rightarrow 4$ (checked condition 6 times)

```
    return s → 1
```

$f(n) = 2n+3$ * time function

$\boxed{O(n)}$

```
def algorithmAdd(a, b, n):
    for i in range(n): → n+1
        for j in range(n): → n+1
            c[i,j] = a[i,j] + b[i,j] → n
```

n : n
3 : 3

$f(n) = 2n^2 + 2n + 1$

$\boxed{O(n^2)}$

space complexity

$a \rightarrow n^2$
$b \rightarrow n^2$
$c \rightarrow n^2$
$n \rightarrow 1$
$i \rightarrow 1$
$j \rightarrow 1$ +

$s(n) = 3n^2 + 3$

$\boxed{O(n^2)}$

* for each nested loop, the time + space complexity gets exponentially bigger such that $O(n^i)$ where i is the number of for loops.

Time complexity of while / conditional:

```
i = 0 → 1
while (i < n): → n+1
  statement → n
  i += 1 → n
```

$f(n) = 3n + 2$

$\boxed{O(n)}$

* the reason for big O notation is that from any perspective other than the computer's, there is no interest or acknowledgeable difference with anything other than log or exponents.

```
a = 1
while (a < b):
  statement
  a *= 2
```

a
1
$1 * 2 = 2$
$2 * 2 = 2^2$
$2^2 * 2 = 2^3$
...
2^k

*terminates when $a \geq b$

$a = 2^k$

$2^k \geq b$

$2k = b$

$k = \log_2 b$

$\boxed{O(\log n)}$

```
while (m != n):
    if (m > n):
        m = m - n
    else:
        n = n - m
```

m^{16}	n^2
14	2
12	2
10	2
8	2
6	2
4	2
2	2

16/2 times executed

or ...

n/2 times

$\boxed{O(n) \text{ while min time is } O(1)}$

Types of Time Functions:

$O(1) \rightarrow$ constant

$f(n) = 2$
$f(n) = 500$
$f(n) = 10000$
$\Big\}\ O(1)$

$O(\log n) \rightarrow$ logarithmic

* log base doesnt matter

$O(n) \rightarrow$ linear

$f(n) = 2n+3$
$f(n) = 500n$
$f(n) = \frac{n}{500} + 6$
$\Big\}\ O(n)$

$O(n^2) \rightarrow$ quadratic

$O(n^3) \rightarrow$ cubic

$O(2^n) \rightarrow$ exponential

$O(3^n)$

$O(n^n)$

Asymptotic Notations:

O big-oh upper bound

Ω big-omega lower bound

Θ theta average bound

O

$f(n) = 2n+3$

$2n+3 \leq \underline{10n}$ some value that makes it true

$\therefore f(n) = O(n)$

$2n+3 \leq 2n^2 + 3n^2$

$\therefore f(n) = O(n^2)$

Comparison of Functions:

obliviously you can test values between the functions and see how the outputs vary. Another method is as follows:

$n^2 < n^3$

apply log on both sides

$\log n^2 \quad \log n^3$

$2\log n < 3\log n$

formulas for log

1. $\log a \cdot b = \log a + \log b$
2. $\log \frac{a}{b} = \log a - \log b$
3. $\log a^b = b \log a$
4. $a^{\log_c b} = b^{\log_c a}$
5. $a^b = n$ then $b = \log_a n$

Practice)

$f(n) = n^2 \log n > g(n) = n(\log n)^{10}$

apply log

$\log[n^2 \log n] \qquad \log[n(\log n)^{10}]$

$\log n^2 + \log\log n \qquad \log n + \log(\log n)^{10}$

$\underline{2\log n} + \log\log n > \underline{\log n} + 10\log(\log n)$

Best, Worst, Average Cases:

Linear Search

search for key element k

8	6	12	5	9	7	4	3	16	18
0	1	2	3	4	5	6	7	8	9

$k = 7$

* if you search in a list, its either the item is found or not found

Best: $B(n)$

searching key is present at first index $\boxed{O(1)}$

Worst: $W(n)$

searching key is at last index $\boxed{O(n)}$

Average: $A(n)$

(all possible case time) ÷ (no. of cases)

$(1+2+3+\ldots+n) \div n$

$$= \frac{\frac{\cancel{n}(n+1)}{2}}{\cancel{n}} = \boxed{\frac{n+1}{2}}$$

How to apply notations*

$B(n) = 1$
$B(n) = O(1)$
$B(n) = \Omega(1)$
$B(n) = \Theta(1)$

since its constant, it belongs to constant class.

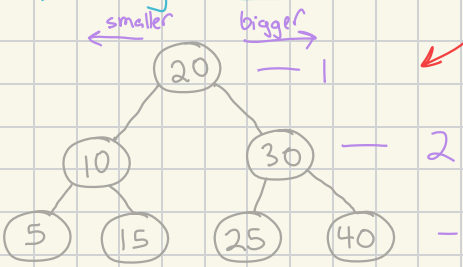
$W(n) = n$
$W(n) = O(n)$
$W(n) = \Omega(n)$
$W(n) = \Theta(n)$

since its n, no fixed notation.

best and worst can be expressed in any notation*

DONT assume that worst case should be upper bound/max, and best is lower bound/omega. They can be expressed by ALL notations.

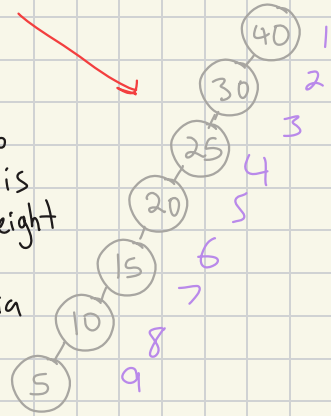
Binary Search Tree



min height $\log(n)$

max height n

*time taken to navigate tree is equal to the height of it; height depends on data structure



Best: $B(n)$

searching for root element $\boxed{O(1)}$

Worst: $W(n)$

searching for leaf element (last row) $\boxed{O(\text{height})}$

min $W(n) = \log n$

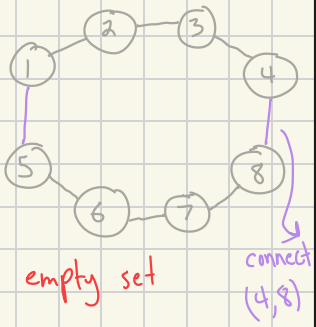
max $W(n) = n$

Disjoint Sets:

*useful for detecting a cycle in a non-directed graph.

$S_1 = \{1, 2, 3, 4\}$

$S_2 = \{5, 6, 7, 8\}$

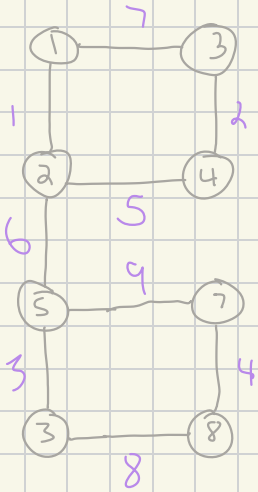


$S_1 \cap S_2 = \emptyset$ or empty set

$S_1 \cup S_2 = S_3 = \{1, 2, 3, \ldots, 7, 8\}$

Operations:

- find is to find which set an element belongs to.
- union is the combination of two or more sets.



$u = \{\cancel{1}, \cancel{2}, \cancel{3}, \cancel{4}, \cancel{5}, \cancel{6}, \cancel{7}, \cancel{8}\}$

$S_1 \{1, 2\}$

$S_2 \{3, 4\}$

$S_3 \{5, 6\}$

$S_4 \{7, 8\}$

taking pairs of sides, moving from universal set to S_n